QNGOptimizer returns TypeError when step method called

https://github.com/PennyLaneAI/pennylane/issues/1154

ROW 127

QNGOptimizer returns TypeError when step method called #1154

New issue

Closed #1638

anthayes92 opened on Mar 19, 2021

- *Expected behavior:* Returns a `list` of updated parameters
- *Actual behavior:* Returns a `TypeError`
- *System information:*
  Platform info: Linux-5.8.0-45-generic-x86_64-with-glibc2.29
  Python version: 3.8.5
  Numpy version: 1.19.5
  Scipy version: 1.4.1

Source code and tracebacks

```
import networkx as nx
import pennylane as qml

edges = [(0, 1), (1, 2), (2, 0), (2, 3)]
graph = nx.Graph(edges)

H_c, H_m = qml.qaoa.min_vertex_cover(graph, constrained=False)

def qaoa_layer(gamma, alpha):
    qml.qaoa.cost_layer(gamma, H_c)
    qml.qaoa.mixer_layer(alpha, H_m)

depth = 5

def circuit(params, **kwargs):

    qml.PauliX(wires=0)
    qml.PauliX(wires=3)

    qml.layer(qaoa_layer, depth, params[0], params[1])

dev = qml.device("default.qubit", wires=4)
cost_function = qml.ExpvalCost(circuit, H_c, dev, optimize=True)
params = [[0.5]*depth,[0.5]*depth]

optimizer = qml.QNGOptimizer(stepsize=0.1)


for _ in range(100):
    params = optimizer.step(cost_function, params)
```

co9olguy on Mar 19, 2021 · edited by co9olguy — Edits — Member

Thanks @anthayes92, could you also share the relevant part of the traceback?
I suspect maybe an issue with `ExpvalCost`, or something to do with your input parameter shape

anthayes92 on Mar 19, 2021 — Contributor — Author

Sure thing, here's the traceback:

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
<ipython-input-1-52d4bb51526b> in <module>
     26
     27 for _ in range(100):
---> 28     params = optimizer.step(cost_function, params)
```

Assignees
No one assigned

Labels
No labels

Type
No type

Projects
No projects

Milestone
No milestone

Relationships
None yet

Development
Add support for batch execution to `qml.metric_tensor`
PennyLaneAI/pennylane

Notifications — Custom
Subscribe
You're not receiving notifications from this thread.

Participants

anthayes92 on Mar 19, 2021 Contributor Author ···

Sure thing, here's the traceback:

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
<ipython-input-1-52d4bb51526b> in <module>
     26
     27 for _ in range(100):
---> 28     params = optimizer.step(cost_function, params)

~/.local/lib/python3.8/site-packages/pennylane/optimize/qng.py in step(self, qnode, x, recompute_tensor, metri
    216             array: the new variable values :math:`x^{(t+1)}`
    217         """
--> 218         x_out, _ = self.step_and_cost(
    219             qnode, x, recompute_tensor=recompute_tensor, metric_tensor_fn=metric_tensor_fn
    220         )

~/.local/lib/python3.8/site-packages/pennylane/optimize/qng.py in step_and_cost(self, qnode, x, recompute_tens
    188             if metric_tensor_fn is None:
    189                 # pseudo-inverse metric tensor
--> 190                 self.metric_tensor = qml.metric_tensor(qnode, diag_approx=self.diag_approx)(x)
    191             else:
    192                 self.metric_tensor = metric_tensor_fn(x)

~/.local/lib/python3.8/site-packages/pennylane/tape/qnode.py in _metric_tensor_fn(*args, **kwargs)
   1013
   1014     def _metric_tensor_fn(*args, **kwargs):
-> 1015         jac = qml.math.stack(_get_classical_jacobian(_qnode)(*args, **kwargs))
   1016         jac = qml.math.reshape(jac, [_qnode.qtape.num_params, -1])
   1017

~/.local/lib/python3.8/site-packages/pennylane/_grad.py in _jacobian_function(*args, **kwargs)
    174
    175         if len(argnum) == 1:
--> 176             return _jacobian(func, argnum[0])(*args, **kwargs)
    177
    178         return _np.stack([_jacobian(func, arg)(*args, **kwargs) for arg in argnum]).T

~/.local/lib/python3.8/site-packages/autograd/wrap_util.py in nary_f(*args, **kwargs)
     18             else:
     19                 x = tuple(args[i] for i in argnum)
---> 20             return unary_operator(unary_f, x, *nary_op_args, **nary_op_kwargs)
     21         return nary_f
     22     return nary_operator

~/.local/lib/python3.8/site-packages/autograd/differential_operators.py in jacobian(fun, x)
     57     vjp, ans = _make_vjp(fun, x)
     58     ans_vspace = vspace(ans)
---> 59     jacobian_shape = ans_vspace.shape + vspace(x).shape
     60     grads = map(vjp, ans_vspace.standard_basis())
     61     return np.reshape(np.stack(grads), jacobian_shape)

TypeError: can only concatenate tuple (not "list") to tuple
```


josh146 on Mar 20, 2021 · edited by josh146 Edits Member ···

Hey **@anthayes92**! I think there are two separate things happening here.

Autograd does not like differentiating nested lists.

Instead of

```
params = [[0.5]*depth,[0.5]*depth]
```

you can write

```
params = np.stack([[0.5]*depth, [0.5]*depth], requires_grad=True)
```

```
params = np.stack([[0.5]*depth, [0.5]*depth], requires_grad=True)
```

This fixes the autograd error you are getting in the exception above.

However, that leads us on to the second problem:

The metric tensor is only defined for the quantum circuit gate arguments, while the optimizer is instead optimizing the QNode arguments.

This is a bit of subtlety, and is caused by the `qml.layer()` function. You have an array of parameters of shape `[2, 5]` as input to the QNode, but due to the repetition of layers, the resulting quantum circuit has 60 trainable gate arguments. As a result, the resulting metric tensor will be of size `[60, 60]`!

```
>>> mt = qml.metric_tensor(cost_function)(params)
>>> print(params.shape, mt.shape)
(2, 5)  (60, 60)
```

This will confuse the optimizer, which won't be able to apply the QNG update step due to the shape mismatch.

The reason it has been implemented like this:

1. The introductory paper https://arxiv.org/abs/1909.02108 does not consider QNG optimization in the context of classical pre-processing of parameters; convergence is only proven when optimizing the gate arguments directly.

2. Historically, PennyLane also did not allow classical pre-processing inside a QNode, so you were 'blocked' from even attempting to do this in the software.

However, since the new core, (2) is no longer the case, as you have shown in your code example above 🙂

Workarounds

I'm not really sure what to do here. We have three options I can think of off the top of my head.

1. Leave the behaviour as-is, and raise a more useful exception in the optimizer.

2. Modify the `qml.metric_tensor` function to take into account classical processing between QNode args and gate args.

This is pretty simple to do. If we say that `f: R^m -> R^n` is the function representing the transformation from QNode arguments to gate arguments, we could simply compute the Jacobian of this function, and return `jac.T @ metric_tensor @ jac`. This is already quite easy in PennyLane:

```
>>> from pennylane.tape.qnode import _get_classical_jacobian
>>> jac = qml.math.stack(_get_classical_jacobian(_qnode)(*args, **kwargs))
>>> mt = qml.metric_tensor(qnode)(*args, **kwargs)
>>> mt = qml.math.tensordot(mt, jac, axes=[-1, 0])
>>> mt = qml.math.tensordot(jac, mt, axes=[0, 0])
```

In fact, I've made this change in the branch `fix-1154` (feel free to check it out and try locally), and your QAOA example trains very well:

```
Cost: -0.40057391269073295
Cost: -0.7268916684780963
Cost: -0.9224609566668688
Cost: -1.0434952436963247
Cost: -1.1458920801395698
Cost: -1.2608435077627338
Cost: -1.4016272161590428
Cost: -1.570999017410614
Cost: -1.7649277793607872
Cost: -1.9744020721362818
Cost: -2.1843779694162007
Cost: -2.375999280893928
Cost: -2.534565991142417
Cost: -2.6552873670133157
Cost: -2.743599940207657
Cost: -2.8083248766962265
Cost: -2.856584386208008
Cost: -2.8930529233848223
Cost: -2.9207394714729924
Cost: -2.9417108541537256
Cost: -2.9574920403418647
Cost: -2.9692628260314633
Cost: -2.9779547882892436
```

```
Cost: -2.8083248766962265
Cost: -2.856584386208008
Cost: -2.8930529233848223
Cost: -2.9207394714729924
Cost: -2.9417108541537256
Cost: -2.9574920403418647
Cost: -2.9692628260314633
Cost: -2.9779547882892436
Cost: -2.984307487473674
Cost: -2.988902676037116
Cost: -2.992192882000369
Cost: -2.9945215574673965
Cost: -2.996143598489102
Cost: -2.997236836771621
Cost: -2.9979120262799204
```

My one concern is if this 'hybrid' QNG optimization is guaranteed to converge better than vanilla gradient descent (or even guaranteed to converge at all), since it is not explored in the paper.

3. The last solution is more radical: provide a keyword argument so that the **natural** quantum gradient of the QNode is returned, rather than just the quantum gradient.

E.g.,

```python
@qml.qnode(dev, natural_gradient=True)
def circuit(params):
    ...
```

The quantum circuit will return `F^{-1} @ quantum_gradient` to the ML library, which will then continue to perform standard backpropagation with the remainder of the classical part of the computation.

This is pretty much equivalent to (2) in practice, but conceptually clearer; the metric tensor is being applied directly to the quantum gradient during backpropagation, rather than to the full hybrid gradient later during optimization. Another advantage is that you can then use *any* optimizer (e.g., `qml.GradientDescentOptimizer`, `qml.AdamOptimizer`), and the quantum natural gradient will continue to be used for the quantum components.

Final comment

After thinking about it more, I'm a fan of solution (3). It is a much more flexible approach, and 'shifts' the burden of applying the metric tensor to the gradient vector *away from* the optimizer gradient update step, and *into the quantum gradient logic*.

Instead of QNG being an optimization approach, it is an extension to the quantum gradient.

In other words, instead of thinking of a 'specific QNG optmizer that only applies to purely quantum systems', you can build hybrid models where quantum components provide the quantum natural gradient during backprop. You can even have multiple QNodes in an optimizaton, with one using the quantum natgrad, and another using regular quantum gradients.

josh146 mentioned this in 2 pull requests on Sep 10, 2021

[WIP] Batch evaluation of the metric tensor #1636

Add support for batch execution to `qml.metric_tensor` #1638

josh146 closed this as completed in #1638 on Sep 15, 2021

Add support for batch execution to `qml.metric_tensor` #1638

Code

Merged josh146 merged 29 commits into `master` from `batch-metric-tensor` on Sep 15, 2021

Conversation 48 | Commits 29 | Checks 0 | Files changed 14 | +384 −270

josh146 commented on Sep 10, 2021 • edited — Member

Context: With the low-level differentiable batch-execution pipeline now available in PennyLane, and the availability of `@qml.batch_transform`, we can now start porting our existing batch-tape transforms to take advantage of this framework.

Description of the Change:

- When using `qml.math.get_trainable_indices()`, only NumPy arrays with `requires_grad=True` are taken as trainable when using Autograd. This is a requirement for batch transformations like the metric tensor to work correctly.
- When using the QNode in backpropagation, trainable parameter indices are computed and stored. Previously, a backpropagation QNode would simply mark *all* parameters as trainable. This extra information is needed for the metric tensor.
- The metric tensor has been converted into a batch transformation, that accepts both tapes *and* QNodes as input. This makes the old `metric_tensor_tape` function irrelevant, and it has been removed. The tests have been likewise updated.
- Previously, `qml.metric_tensor(qnode)(*args, **kwargs)` would only return the metric tensor with respect to gate arguments, and ignore any classical processing inside the QNode, even very trivial classical processing such as parameter permutation. This would lead to many reported user bugs, such as QNGOptimizer returns TypeError when step method called #1154. In this new framework, the metric tensor now takes into account classical processing, and returns the metric tensor with respect to *QNode arguments*, not simply *gate* arguments.

  To revert to the previous behaviour of returning the metric tensor with respect to gate arguments, `qml.metric_tensor(qnode, hybrid=False)` can be passed.

Benefits:

- The `qml.metric_tensor()` function now makes use of batch execution for submission of circuits required for metric tensor computation.
- The `qml.metric_tensor()` function now takes into account classical computation inside the QNode, for example, if gate arguments are repeated or permuted.
- QNodes in backpropagation mode correctly track trainable parameters, fixing an issue that was long part of our test suite.

Possible Drawbacks: n/a

Related GitHub Issues: Closes #1154

Reviewers
anthayes92 ✓
glassnotes ✓

Assignees
No one assigned

Labels
review-ready 👌

Projects
None yet

Milestone
No milestone

Development
Successfully merging this pull request may close these issues.
QNGOptimizer returns TypeError when step ...

Notifications — Customize
Subscribe
You're not receiving notifications from this thread.

3 participants